

UF1

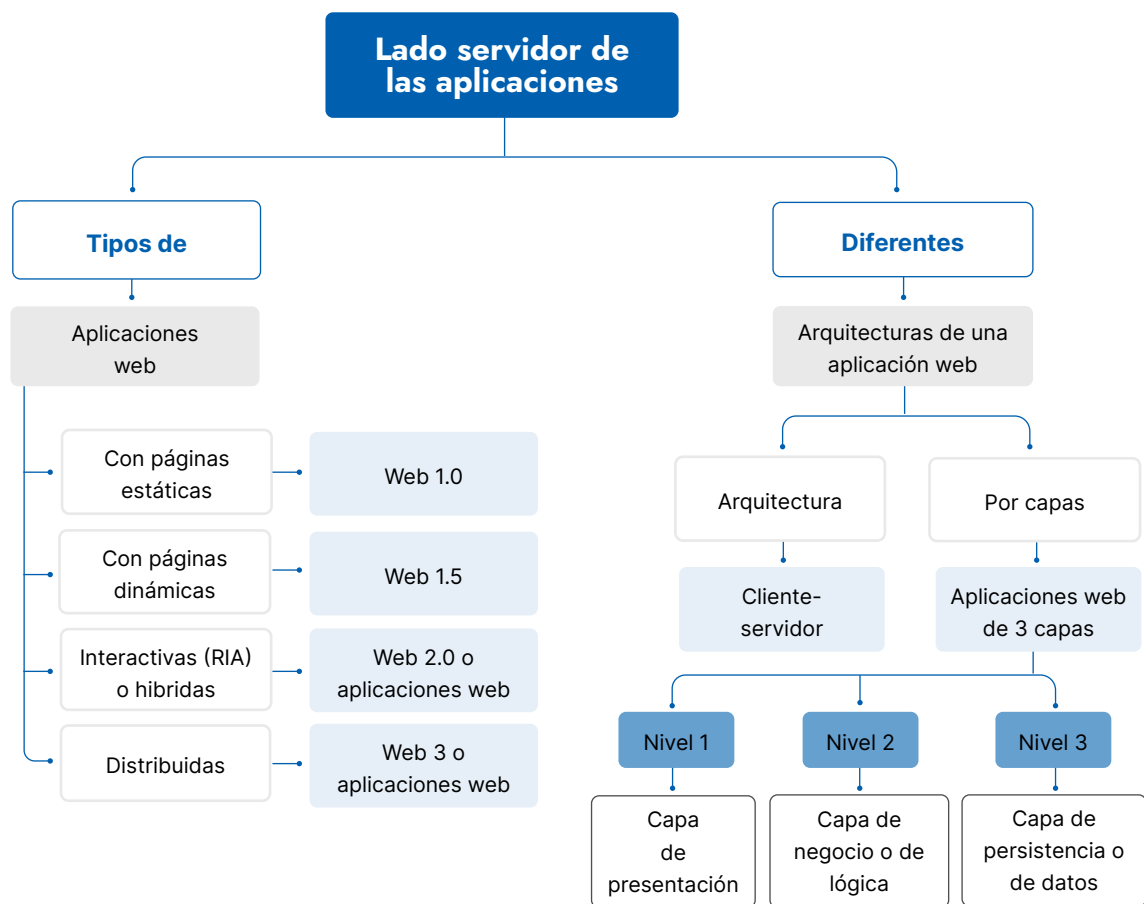
Introducción al lado servidor de las aplicaciones web

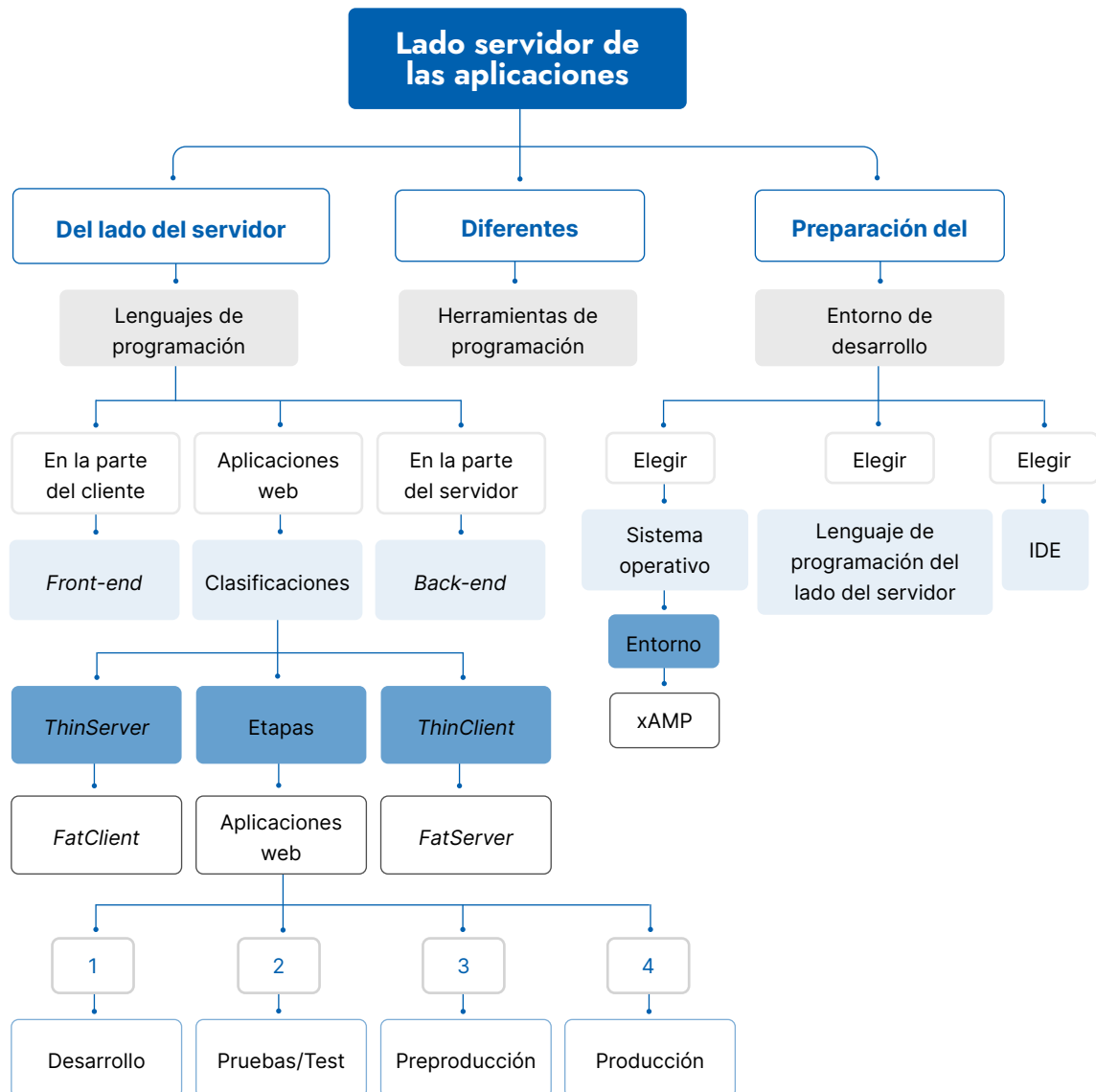
Desarrollo web en entorno servidor

ÍNDICE

Mapa conceptual	03
1. Tipos de aplicaciones web	05
1.1. Web 1.0 con páginas estáticas	06
1.2. Web 1.5 con páginas dinámicas	07
1.3. Web 2.0 o aplicaciones web interactivas (RIA) o híbridas.....	08
1.4. Web3 o aplicaciones web distribuidas.....	09
2. Arquitecturas de las aplicaciones web	11
2.1. Aplicaciones web cliente-servidor.....	11
2.2. Aplicaciones web de tres capas.....	13
3. Lenguajes de programación del lado del servidor	15
3.1. En la parte del cliente, denominada <i>front-end</i>	15
3.2. En la parte del servidor, denominada <i>back-end</i>	16
A. Lenguajes de programación del lado servidor.....	16
3.3. Clasificaciones de las aplicaciones web	17
3.4. Etapas de las aplicaciones web	17
3.5. Herramientas de programación.....	18
4. Preparación del entorno de desarrollo	20
4.1. Sistema operativo	20
4.2. Lenguaje de programación del lado del servidor	21
4.3. IDE	21
4.4. xAMP.....	22
A. Apache.....	22

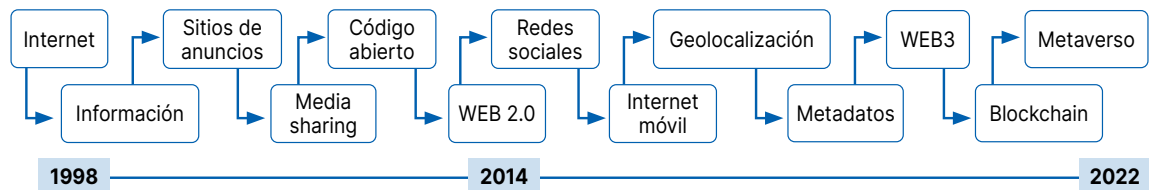
Mapa conceptual





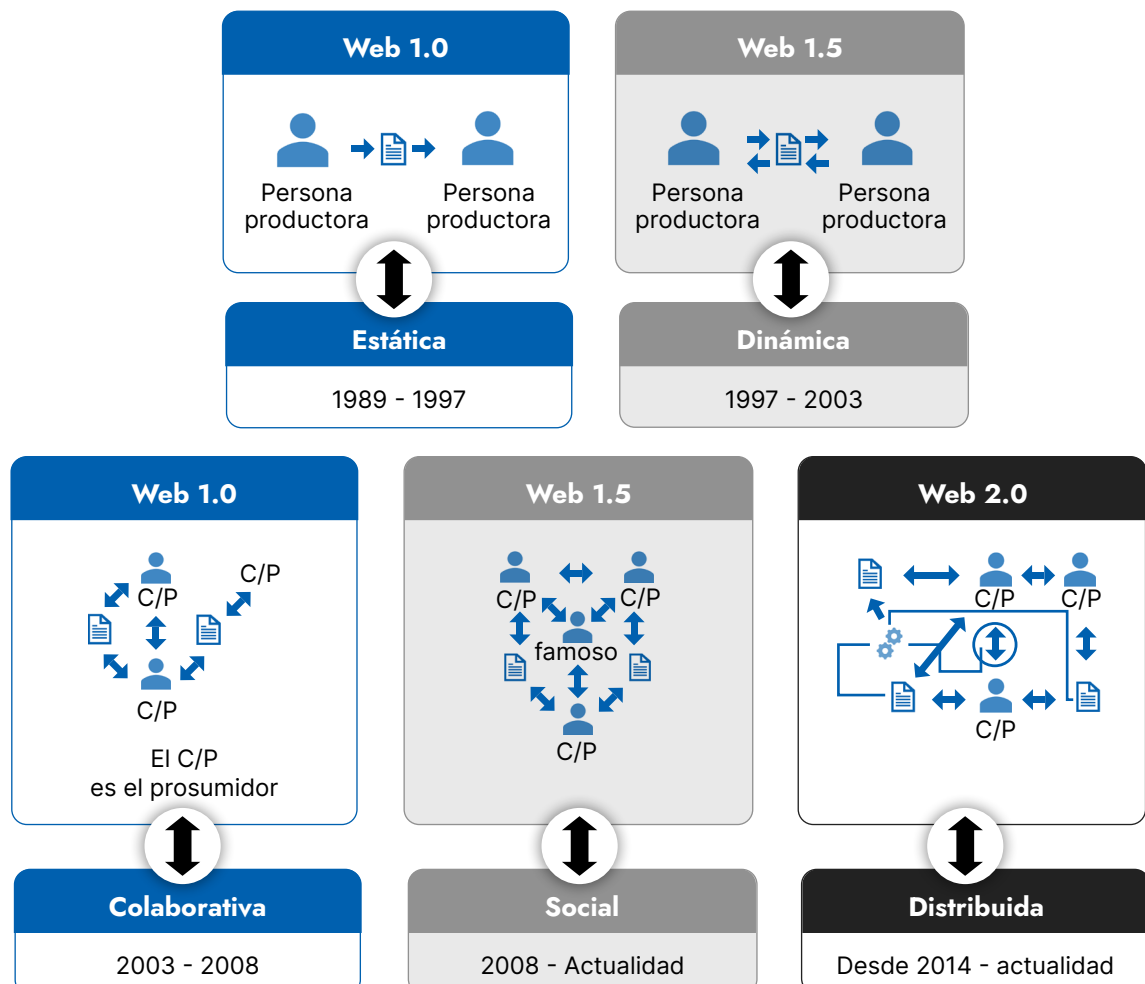
01 Tipos de aplicaciones web

Para entender mejor las aplicaciones web actuales, siempre viene bien conocer un poco de historia de la evolución de estas. Para ello, empezaremos por la primera versión, del siglo xx, que se denominó web 1.0, para llegar hasta la actual web3.



Evolución de Internet hasta la web3

La evolución de las tecnologías ha permitido a los usuarios avanzar desde los inicios de la web, pasando por una Internet más social, hasta llegar a la Internet distribuida que tenemos actualmente, donde se está diluyendo la frontera entre la vida real y la vida virtual, para fusionarse en una única vida en dos formatos: analógica y digital.



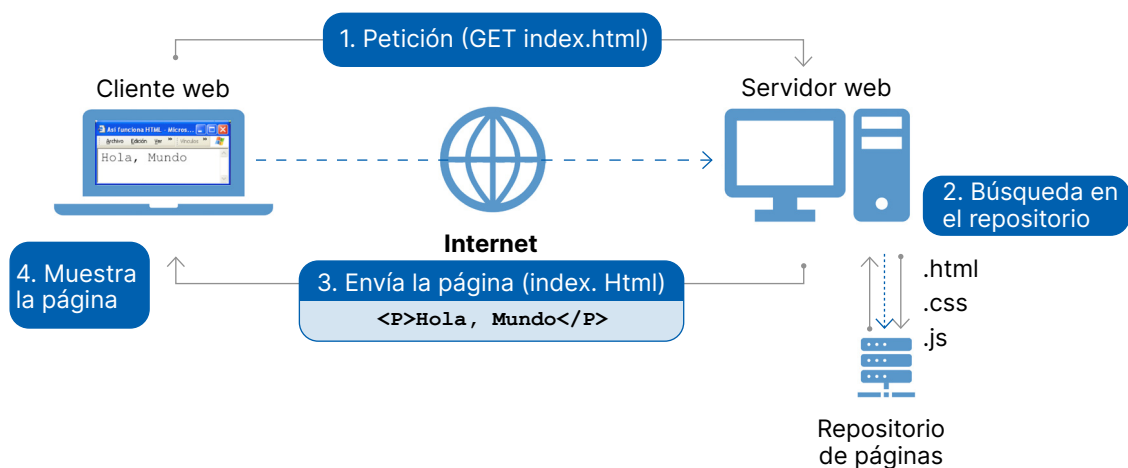
Uso de los usuarios de las nuevas aplicaciones web surgidas en cada etapa

1.1. Web 1.0 con páginas estáticas

Esta etapa duró aproximadamente ocho años, hasta la aparición del HTML 4.0 en 1998. Para situarnos en aquella época, debemos saber qué tecnologías de telecomunicaciones estaban disponibles. Durante aquellos años, aparecen de las primeras conexiones de acceso conmutado, tanto fijo como móvil (RTC, RDSI, GSM/2G, GPRS/2G).

También surgen las primeras etiquetas multimedia del estándar HTML, que admiten la incorporación de pequeños programas, denominados *applets*, realizados con el lenguaje de programación Java para ejecutarlos en la parte cliente.

Mientras, en la parte servidor solo se podían ejecutar programas con una **interfaz de entrada común**, que eran denominados **CGI** (*common gateway interface*). Estos podían estar hechos en cualquier lenguaje de programación, mayoritariamente con el lenguaje de programación **C**, aunque se podían hacer con otros, para responder a peticiones concretas. Esta tecnología CGI, y su actualización FastCGI, está en desuso actualmente.



Esquema básico de un servicio web en la etapa 1.0.

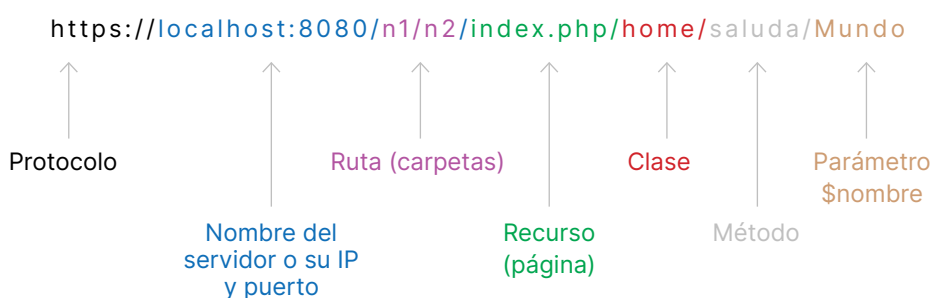
Cuando un cliente realiza una petición de un recurso de una URL al servidor web, este le mostrará una página web estática. Dicho recurso se busca en el sistema de archivos del servidor web y, una vez encontrado, se envía al cliente que lo había solicitado. Toda esa comunicación se lleva a cabo mediante el protocolo HTTP, a través del puerto 80 (que es su valor por defecto), aunque se puede configurar cualquier otro puerto. Por ejemplo, podríamos utilizar el 8080, por cuestiones de seguridad.

El código de estas páginas estáticas contiene básicamente etiquetas del lenguaje HTML, junto con código de etiquetas de estilo en CSS, que le dan el diseño o el aspecto, y puede contener también algo de código escrito con el lenguaje de programación JavaScript, que se ejecuta en el cliente. Es la única opción que le permitía interactuar con el usuario para poder realizar determinadas acciones básicas, como, por ejemplo, validar formularios.

1.2. Web 1.5 con páginas dinámicas

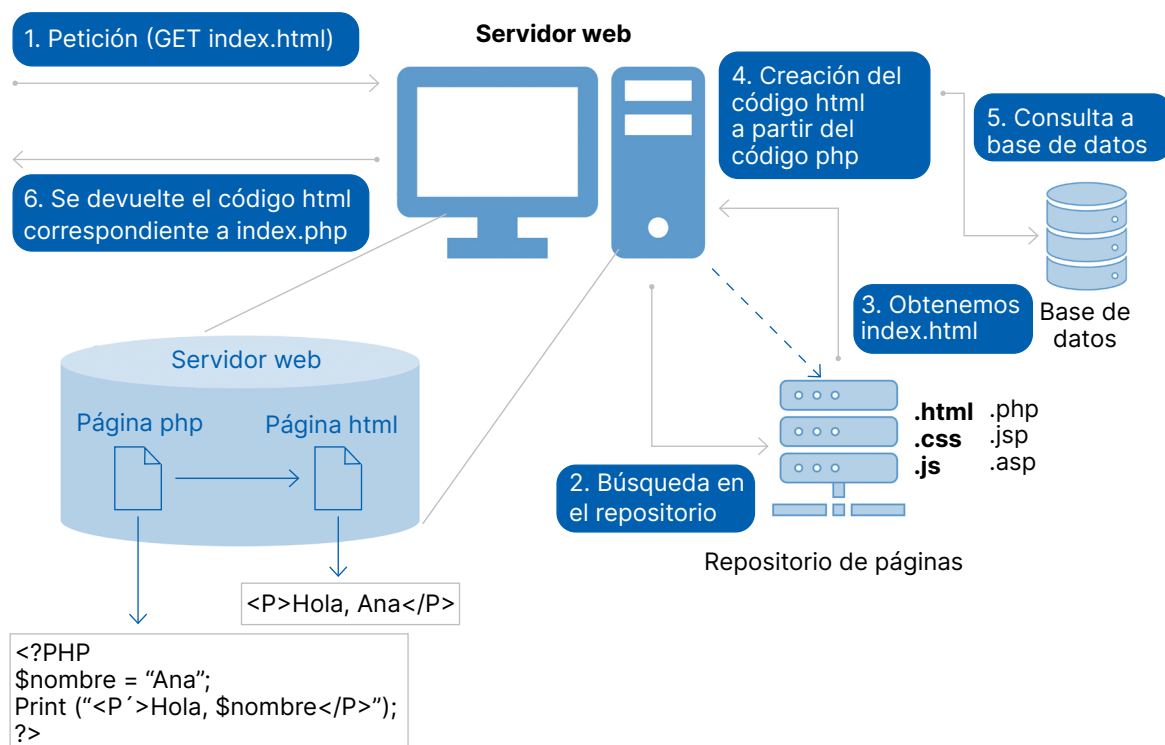
Estas primeras páginas web dinámicas se utilizaron durante seis años, hasta 2003. Estaban escritas en lenguajes de programación del lado servidor, como PHP, ASP.net o JSP, que se ejecutan en el servidor de aplicaciones web.

El servidor web procesará la petición e irá construyendo la página web en HTML de forma dinámica para contestar al cliente.



Elementos de una URL completa que podemos encontrarnos cuando utilizamos un framework.

Las páginas web dinámicas se personalizan, y adaptan la información que muestran a partir de la información recibida en la misma petición o mediante consultas a bases de datos. Lo habitual será que una misma página devuelva resultados diferentes para usuarios diferentes, según los parámetros recibidos.



Esquema básico de funcionamiento del servicio web con páginas dinámicas en la etapa 1.5.

Las páginas se pueden generar dinámicamente con dos modos diferentes de petición del cliente:

- » Modo **POST**: sin necesidad de modificar la URL. Aporta discreción, al no guardar información en el cliente, y flexibilidad, al permitir enviar textos cortos, pero también podemos enviar imágenes o vídeos.
- » Modo **GET**: modifica la URL para añadir tantos pares de variable o valor como hagan falta. Detrás de la URL añadiremos esa información como texto claro, detrás de un símbolo de interrogación «?». Por ejemplo, si pedimos al buscador de Google que nos muestre información de McGraw-Hill, generaría la URL **https://www.google.es/search?q=McGraw-Hill**, donde **q** es la variable y, detrás del igual, su valor; en este caso, «McGraw-Hill».

1.3. Web 2.0 o aplicaciones web interactivas (RIA) o híbridas

Se crearon a partir del comienzo del siglo **xxi** (en concreto, desde 2003 hasta 2014), en dos etapas:

- » Hasta 2008, en la etapa denominada Web 2.0: la de la Internet de las redes sociales, con la aparición de tecnologías de acceso dedicado fijo (ADSL, módem-cable, fibra óptica) o móvil (UMTS/3G, LTE/4G o 5G), que ampliaron el ancho de banda y consiguieron conexiones a Internet más potentes y asequibles.
- » Hasta 2014; la llegada del estándar HTML5 dio por finalizada esta etapa, durante la cual aparecieron nuevas especificaciones estándares del W3C, como fueron XML, JSON, SOAP, UDDI, WSDL, RSS, ATOM o AMF. Esto permitió utilizar estas aplicaciones interactivas de Internet enriquecidas (o RIA, acrónimo del inglés *rich Internet application*), lo que dio lugar a la parte más social (*social media*), con la aparición de las primeras redes sociales, mucho más potentes, que han desatado una nueva guerra de complementos con nuevas funciones multimedia, como la reproducción de vídeos, gráficos vectoriales, animaciones e interactividad para controlar el mercado, lo que ha provocado una auténtica revolución multimedia en la web.

Las tecnologías Flash, Flex y AIR, de la compañía Adobe, fueron las más utilizadas durante esta etapa. Su presencia en el lado del cliente, en los navegadores y en el servidor era habitual en la mayoría del mercado, pero actualmente han desaparecido, y en 2022 ya no se utilizan. Otras tecnologías equivalentes aparecidas en la misma época, como JavaFX de Oracle (no es tecnología web) o Silverlight de Microsoft, fueron sus competidoras naturales en aquel momento.

Las grandes compañías de Internet, entre otras muchas, lideradas por las cinco tecnológicas denominadas GAFAM (Google, Amazon, Facebook, Apple y Microsoft) o FAMGA, trabajaron para crear otros estándares del W3C, como AJAX y HTML5. Consiguieron que no fuesen necesarios complementos no estándares de terceras empresas, como los de Adobe. De este modo, se terminó con esta guerra de complementos en los navegadores, y con AJAX se pudo conseguir un diálogo fluido real entre el cliente y el servidor que permite crear aplicaciones web híbridas más interactivas y que posibilitan trabajar en remoto, como si se estuviera en local.

SerieNum	Fecha	Nombre	Resumen	DNI/CIF	Total	Total IVA
A22-003	01/03/2022	Cliente 1	MARZO [D]	B000111222	400,00	408,00
A22-002	28/02/2022	Cliente 2	FEBRERO [D]	B111222333	400,00	408,00
A22-001	03/01/2022	Cliente 3	ENERO	B222333444	400,00	408,00
Totales					1.200,00	1.224,00
					Suplidos:	0,00
					Total:	1.224,00

Aplicación web AdminLTE, creada con Bootstrap para el frontend y con el framework CodeIgniter de PHP para el backend.

1.4. Web3 o aplicaciones web distribuidas

El futuro va hacia una Internet distribuida, más libre y menos controlada por las grandes corporaciones de Internet, es decir, más autónoma. Los gobiernos quieren restar poder a las grandes tecnologías y aumentar la privacidad de la ciudadanía.

La evolución es continua y parece no tener fin. La idea es superar las barreras actuales de los usuarios *prosumidores*, que son grandes productores y consumidores de contenidos (sean estos contenidos creados por ellos o no), pero sin control sobre ellos. Según las licencias abusivas, han de cederlos a las empresas donde los publican, por lo que pierden su autoría, y, por tanto, su poder.

Esta nueva Web3 distribuida va a ir potenciando el uso del *token* digital (o vale digital) para controlar qué compartimos y con quién, sin perder la autoría y el control sobre ellos. Estas nuevas posibilidades llegarán con las nuevas tecnologías que se están creando. Se irán explotando hasta 2030, cuando llegarán a su cenit, con la aparición del 6G, que nos permitirá comunicaciones móviles holográficas.

Todo lo que conocemos está cambiando; todos los servicios actuales van a ir migrando a la Web3. Es una tecnología disruptiva, que brindará grandes oportunidades a los programadores que desarrollen estas nuevas aplicaciones web distribuidas, usando la interfaz Web3 con todas estas novedades, como el Metaverso de Facebook u otros mundos virtuales paralelos de otras empresas, sin olvidar las redes distribuidas de criptomonedas como Bitcoin o Ethereum, que están surgiendo como alternativa al dinero digital oficial de los países.

1

CASO PRÁCTICO

**ANALIZANDO APLICACIONES**

Investiga sobre aplicaciones realizadas con ILE RPG para el sistema AS400 de IBM, con su sistema operativo OS/400, e indica las ventajas y desventajas que presentan sobre otras aplicaciones más «modernas».

```
MAIN                               Menú principal OS/400                      Sistema:  S1057
Seleccione una de las opciones siguientes:

1. Tareas de usuario
2. Tareas de oficina
3. Tareas generales del sistema
4. Archivos, bibliotecas y carpetas
5. Programación
6. Comunicaciones
7. Definir o cambiar el sistema
8. Manejo de problemas
9. Visualizar un menú
10. Opciones de Information Assistant
11. Tareas de Client Access/400

90. Finalizar la sesión

Selección o mandato
==>

F3=Salir  F4=Solicitud  F9=Recuperar  F12=Cancelar
F13=Information Assistant  F23=Establecer menú inicial
(C) COPYRIGHT IBM CORP. 1980, 2002.
```

SOLUCIÓN

Las aplicaciones en modo texto fueron las primeras que surgieron con los primeros grandes ordenadores, denominados *mainframes*, en las décadas de 1960 y 1970. Hoy en día, se siguen utilizando en muchas grandes empresas de la banca o de la industria, que siguen utilizando código de este tipo con miles de millones de líneas que aún no han migrado a aplicaciones web.

Las aplicaciones han ido evolucionando a una arquitectura cliente-servidor hasta la actualidad, donde existen aún sistemas propietarios como el AS400 de IBM, que siguen manteniendo y utilizando programas hechos con lenguajes de programación del siglo pasado, como ILE RPG o incluso COBOL, que solo funcionan sobre su sistema operativo OS/400. Muchas grandes empresas de la banca o de la industria siguen utilizando código de este tipo, con miles de millones de líneas. Este código se ha de mantener, e incluso hay que crear nuevas funcionalidades que van surgiendo por cambios en la legislación o en la organización de la propia empresa. La inversión hecha en servidores suele estar ya amortizada, pero, al ser sistemas críticos, no es fácil migrarlos. Como aspecto positivo, requieren clientes livianos sin grandes requisitos de hardware, que son fáciles de actualizar, mantener y utilizar.

La diferencia respecto a los clientes que vamos a analizar es que utilizan **protocolos de comunicación** y **formatos de intercambio de información propietarios**, que no siguen ningún estándar y que obligan a estar atados a este proveedor de por vida, hasta que vayan migrándolas a tecnologías del siglo XXI con otro tipo de aplicaciones.

02 Arquitecturas de las aplicaciones web

2.1. Aplicaciones web cliente-servidor

Las aplicaciones cliente-servidor están divididas en dos niveles o capas. Ambas funcionarán a través de la Internet, y la parte cliente irá solicitando recursos a la parte servidor. En la URL podemos utilizar diferentes protocolos de comunicaciones de la familia TPC/IP, entre los que se encuentran FTP, HTTP y HTTPS, y SMTP, entre otros. A su vez, necesitaremos utilizar otros protocolos de la misma familia, como IP, TCP, UDP o DNS, entre otros, para poder funcionar correctamente.

Por ejemplo, el protocolo DNS es un servicio de una base de datos distribuida que contiene la información de cada dominio. Su función principal es dar la IP pública de un dominio. Su gestión está delegada sobre diferentes empresas a nivel mundial. En concreto, los dominios de dos letras de cada país en cada continente o región (ccTLD, *country code top-level domains*) son delegados en un registro regional de Internet (RIR). Solo existen cinco, y son los siguientes:

- » En Europa, lo gestiona el Réseaux IP Européens Network Coordination Center (RIPE NCC).
- » En América del Norte, lo gestiona el American Registry for Internet Numbers (ARIN).
- » En Asia, lo gestiona el Asia-Pacific Network Information Center (APNIC).
- » El África, lo gestiona el African Network Information Center (AFRINIC).
- » El Latinoamérica, lo gestiona el Latin America and Caribbean Network Information Center (LACNIC).

A su vez, cada RIR lo delega para cada país en su centro de información de red (NIC, *network information center*). En nuestro país, el dominio .es está gestionado una entidad nacional, www.nic.es, dependiente de la Administración, que actualmente se reenvía a la web <https://www.dominios.es>. Esta, finalmente, permite que empresas del sector informático sean las que vendan estos dominios a sus clientes o usuarios finales.

En todo el mundo, hay miles de servidores DNS escuchando por su puerto 53 (a través de TCP o de UDP) las peticiones de los millones de clientes que requieran conocer la IP pública del dominio al que quieren acceder para encontrar dicho servidor web.

El servidor web es un proceso pasivo, ya que solo contesta si algún cliente le solicita algún recurso. Dichas solicitudes llegan, mayoritariamente, a través de los protocolos HTTP/HTTPS.

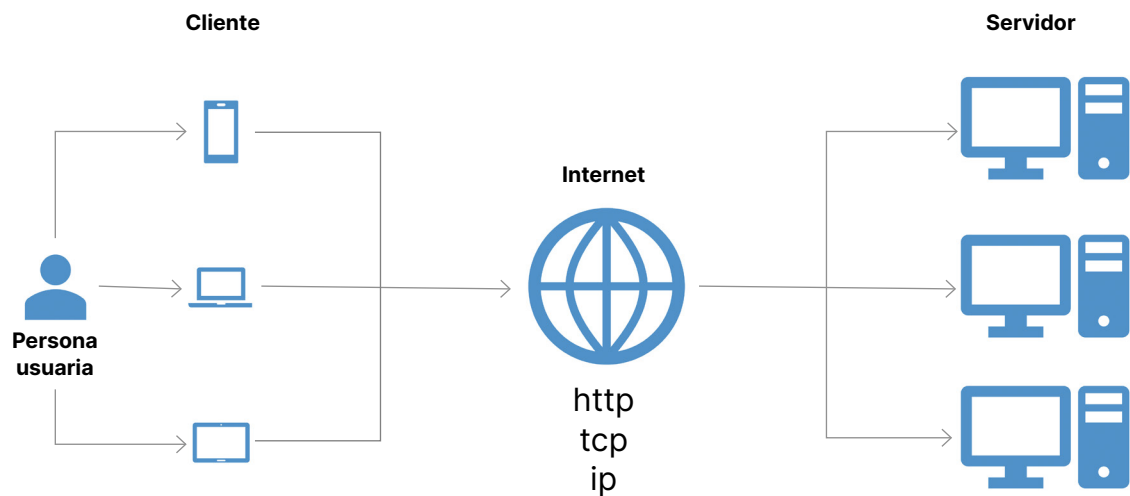


Evolución del protocolo HTTP

Los servidores web más utilizados en el mercado que podemos instalar son:

- » Los privativos, como el Internet Information Services (IIS), de Microsoft, o el WebLogic Server, de Oracle.
- » O los proyectos de código abierto (*open source*), como el Servidor HTTP Apache, el Lighttpd o el servidor web/proxy NginX.

Mientras tanto, en la otra capa tenemos a los clientes activos, que son los consumidores de servicios de la parte servidora. El usuario interactuará a través de las aplicaciones web, para ir pidiendo la información que va necesitando visualizar en cada momento.



Esquema de funcionamiento de una arquitectura web de dos capas cliente-servidor.

La arquitectura cliente-servidor tiene sus pros y sus contras; vamos a ver algunos de ellos:

VENTAJAS	DESVENTAJAS
» Puede ser más escalable, tanto verticalmente como horizontalmente, si lo tenemos en la nube en la parte servidor. Por tanto, podemos aumentar el servicio que podemos ofrecer a más clientes. » Mayor control, al estar centralizado en la parte servidor, lo que nos permite tener un mantenimiento más cómodo y fácil solo en esta parte. » Las tecnologías para los servidores de aplicaciones web han evolucionado para ofrecer una experiencia de usuario más amigable (UX/UI), facilitando su uso, sin olvidar la seguridad al utilizar transacciones y sistemas de bloqueo sobre nuestros datos.	» Al incrementar el tráfico en la red, requiere tolerancia a fallos, añadiendo más hardware con otro servidor secundario, que es un espejo del primario.

2.2. Aplicaciones web de tres capas

Son una evolución más profesional de la arquitectura cliente-servidor. Esta nueva arquitectura implementa una división mayor de funcionalidades en las diferentes capas; en concreto, **el servidor se desdobra** para trabajar con estas tres capas:

**Capa de presentación
(nivel 1):**

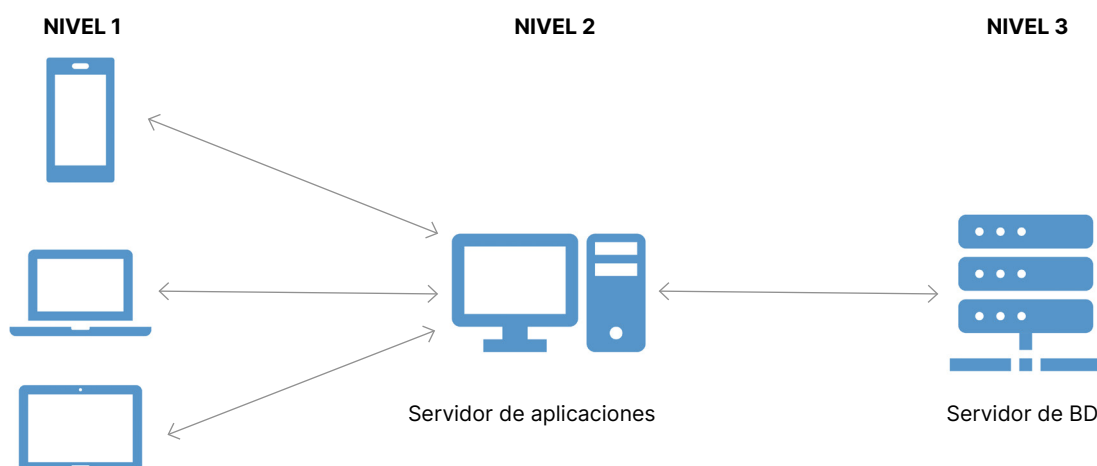
Es el *front-end* que ven y utilizan los usuarios o clientes, que ha de verse bien en todos los navegadores existentes del mercado de manera adaptativa (*responsive*) con diferentes tamaños de pantalla, en todo tipo de dispositivos electrónicos, desde wearables hasta *smartTV*, pasando por tabletas u ordenadores personales.

**Capa de negocio o
de lógica (nivel 2)**

Es el *back-end*, es decir, la capa que conoce y gestiona las funcionalidades que esperamos del sistema o aplicación web. Conoce el modelo o la lógica del negocio o servicio que queremos dar; corresponde al **servidor de aplicaciones**. No obstante, cada vez existen más tendencias de dividirlo en microservicios, serverless o servicios en la nube. Aún se sigue implementando, como haremos nosotros en este módulo.

**Capa de persistencia
o de datos (nivel 3):**

Es la capa donde residen los datos y la encargada de acceder a ellos. En este modelo de tres capas, reside en un servidor específico y especializado para ella. Permite centralizar los datos, y que estos sean accesibles desde más de uno o varios servidores web, y puede tener un conjunto de servidores web (clúster) que puedan balancear la carga si nuestra aplicación web recibe miles o millones de peticiones concurrentes cada segundo o minuto. E incluso podemos disponer de bases de datos distribuidas en varios servidores de datos por diferentes ubicaciones para que estos estén lo más cerca de los usuarios finales. Básicamente, su función es entregar los datos que le pida la capa intermedia, es decir, el servidor de aplicaciones.



Esquema de funcionamiento de una arquitectura web de tres capas.

La arquitectura en tres niveles tiene sus pros y sus contras; veamos algunos de ellos:

VENTAJAS	DESVENTAJAS
» Un mayor grado de flexibilidad, al delegar en cada capa diferentes tareas. » Una mayor seguridad, al definir la arquitectura independientemente para cada servicio y en cada nivel. » Un mejor rendimiento, ya que las tareas se comparten entre servidores. » Una mayor reutilización de la capa de lógica intermedia por múltiples aplicaciones, si tiene un buen diseño modular. » Una mayor facilidad para cambiar o modificar parte de una capa sin que se vean afectadas las restantes capas.	» Al incrementar el tráfico en la red, requiere balancear la carga e introducir tolerancia a fallos. » Los navegadores actuales pueden variar y no ser compatibles al 100 %. » La estandarización a nivel de tecnologías o proveedores tarda en evolucionar o desarrollarse.

Si trasladamos esta arquitectura o modelo de tres capas a una grande o mediana empresa (GYME), a una consejería/ministerio o a toda la Administración, se puede dar servicio a miles o incluso a millones de clientes si lo abrimos a los ciudadanos. Así, podemos encontrarnos cientos o miles de servidores web especializados y virtualizados, cada uno de ellos dedicado a dar servicio en su nivel o capa, de forma que se puede disponer de infraestructura de respaldo.

03 Lenguajes de programación del lado del servidor

Actualmente, los desarrolladores de aplicaciones web híbridas programan dos partes: la parte del cliente, denominada *front-end*, orientada a atender a los clientes, y la parte del servidor, denominada *back-end*, orientada a administrar y generar contenidos.

3.1. En la parte del cliente, denominada *front-end*

El **front-office (front-end)** es el conjunto de tecnologías que forman la capa de presentación. Es la parte que verán la gran mayoría de usuarios de nuestras aplicaciones web (usuarios externos).

Está desarrollada con HTML5, CSS3 y JavaScript, aunque puede usar algún *framework* de programación reactiva, como Angular, Vue.js, React, etcétera.

En su desarrollo participan los programadores, los diseñadores web y los —cada vez más valorados— diseñadores de UI/UX, es decir, los técnicos que se encargarán de cuidar la interfaz de usuario (UI) y crear una mejor experiencia del usuario (UX).

3.2. En la parte del servidor, denominada *back-end*

Back-office (back-end) es el conjunto de tecnologías que forman la capa de negocio, con la lógica que queramos implantar. Es la que utilizan las personas que producen el contenido, así como los técnicos informáticos que las administran y configuran (usuarios internos). Es desde donde se accede a la capa de persistencia de los datos, es decir, al SGBD integrado en este. Está desarrollado con lenguajes de servidor como PHP, Perl, Python, Ruby, Java EE, C# o ASP.net de Microsoft, entre otros.

A. Lenguajes de programación del lado servidor

El lenguaje de programación del lado del servidor es código interpretado o precompilado como objeto (dependiendo del lenguaje de programación utilizado), que es ejecutado por un software específico en la parte del servidor.

Podemos utilizar diferentes tipos:

LENGUAJES DE SCRIPTING

Son una familia de lenguajes de programación que simplifican el procesamiento de texto; son ideales para la generación dinámica de páginas HTML.

- » PHP (PHP hypertext processor).
- » ASP (active server pages) clásico.
- » JSP (Java server pages).
- » U otros lenguajes, como Perl o Python, entre otros.

APLICACIONES CON SERVLETS

Son una clase de Java que recibe y maneja peticiones HTTP/HTTPS. Son muy utilizadas como base de comunicaciones en *frameworks* como Spring para Java.

**APLICACIONES
HÍBRIDAS
DE CÓDIGO
REPARTIDO**

- » Tecnología intermedia con respecto a las anteriores.
- » La plataforma Microsoft .Net Framework, a través de ASP.Net, sería la más representativa.
- » Los formularios web están contenidos en archivos con una extensión ASPX, que son los que el cliente solicita a través de una URL al servidor.
- » Estos ficheros ASPX contienen código HTML o estático y también etiquetas propias de la plataforma .Net.
- » Estas etiquetas definen controles web, que se procesan del lado del servidor, y controles de usuario, donde los desarrolladores colocan todo el código estático y dinámico requerido por la página web

3.3. Clasificaciones de las aplicaciones web

Existen diferentes clasificaciones de los modelos de programación web híbrida:

- » Según el tamaño de los componentes:
 - *FatClient (ThinServer)*: el mayor peso de la aplicación se ejecuta en el cliente.
 - *FatServer (ThinClient)*: la funcionalidad asociada al cliente está limitada a la presentación de la información enviada por el servidor.
- » Según la naturaleza del servicio ofrecido: en función de las capacidades ofrecidas por el servidor.
 - Servidores de archivos, servidores de bases de datos, servidores de transacciones, servidores de objetos, servidores web, etcétera.
- » Reparto de funciones entre cliente y servidor: gestionar y distribuir las responsabilidades de cada una de las prestaciones funcionales entre el cliente y el servidor.

Pero veamos en más detalle las diferentes arquitecturas que nos podemos encontrar, dependiendo de dónde vamos a tener cada una de estas partes.

3.4. Etapas de las aplicaciones web

Durante el desarrollo de una aplicación web híbrida podemos pasar por varias etapas o entornos. Vamos a ver cuáles son.

ETAPAS	TAREAS
Desarrollo	Los profesionales de TI implementan las nuevas funcionalidades.
Pruebas/test	Se validan y se prueban las nuevas funcionalidades.
Preproducción	Se realizan pruebas funcionales y de garantía. Se empieza a dar formación a los futuros usuarios.
Producción	Se despliegan las aplicaciones web en sus diferentes entornos.

Tabla 1. Diferentes etapas de desarrollo web.

3.5. Herramientas de programación

El proceso de desarrollo no tiene por qué realizarse en el mismo equipo en el que finalmente se despliegue y ejecute la aplicación web que se está desarrollando.

Instrumentos involucrados en el desarrollo web

Entornos de programación: programas, aplicaciones o simples utilidades destinadas a la programación web

- » Navegadores.
- » Editores de documentos.
- » Entornos de programación.
- » Herramientas de tratamiento de imágenes.
- » Herramientas para la creación y administración de bases de datos.
- » Marcadores de texto: nos ayudan a identificar mejor la sintaxis del lenguaje, cambiando de color las etiquetas, realizando tabulaciones en el texto, etcétera.
- » Notepad++, UltraEdit, Vim, etcétera.
- » Herramientas genéricas: ofrecen capacidades tales como la sugerencia de estructuras o funciones predeterminadas o la posibilidad de validar la corrección del código escrito.
- » Microsoft Expression Web, que sustituyó a FrontPage, Adobe Dreamweaver, etcétera.
- » Herramientas específicas: para una tecnología concreta; exigen que el desarrollador tenga instalada una plataforma de desarrollo concreta.
- » Visual Studio Code (VSC), Eclipse, NetBeans, etcétera.

2 CASO PRÁCTICO



FRAMEWORK

Un *framework* es un conjunto de bibliotecas de código que nos permite desarrollar aplicaciones web más ágilmente, lo cual redundará en una mayor rapidez en su desarrollo. Investiga qué *frameworks* son los más utilizados para los lenguajes de programación con más uso en la actualidad.



SOLUCIÓN

Estos son los *frameworks* más utilizados actualmente. Nosotros nos centraremos en los que pueden utilizar PHP y, sobre todo, profundizaremos en el uso de Laravel.

04 Preparación del entorno de desarrollo

El entorno de desarrollo necesario para trabajar programando aplicaciones web constará de varios elementos o tecnologías. Estos elementos deben estar instalados en nuestro equipo de trabajo.

Por ejemplo:

- » **IDE:** puede venir condicionado por el lenguaje de programación y la versión que se vaya a utilizar.
- » **Plataforma de pruebas xAMP:** dependiente del sistema operativo que se vaya a utilizar.

Necesitamos saber la carpeta donde hemos de situar nuestros programas para poder validarlos desde el navegador. La plataforma de pruebas nos permitirá conocer cómo solucionar algunos de los problemas que podemos tener con las diferentes variables de configuración de Apache o del propio complemento de PHP.

Más adelante veremos que, para trabajar con *frameworks*, debemos utilizar un gestor de dependencias, como puede ser **Composer**, según el lenguaje que vayamos a utilizar. Y tendremos en cuenta si vamos a ir guardando copias de la evolución de nuestro código en gestores de versiones como **Git** y plataformas como GitHub, GitLab u otras que os pueda indicar vuestro docente.

4.1. Sistema operativo

Al poner en explotación una aplicación web, deberemos de tener cierto software preinstalado y configurado.

- » En el lado cliente, no nos importará con qué sistema operativo vayamos a trabajar: GNU/Linux, macOS, Windows o Android, iOS o cualquier otro que se utilice; el navegador hará posible su utilización.
- » En el lado servidor, nos centramos en las diferentes maneras disponibles para instalar una plataforma web xAMP: LAMP, MAMP o WAMP.
 - En local en nuestra máquina de desarrollo.
 - En remoto en alguna otra que haga de servidor de pruebas.
 - Con contenedores, podemos optar por la utilización del tipo LXC, Docker u otros.

4.2. Lenguaje de programación del lado del servidor

Vamos a utilizar el lenguaje de programación del lado del servidor denominado PHP (*PHP hypertext processor*). En concreto, utilizaremos su última versión estable en <https://www.php.net/downloads> (que es la php-8.1.6 a 12 de mayo de 2022).

Deberemos utilizar la extensión correcta para el servidor web que vayamos a utilizar; en nuestro caso, vamos a trabajar con la extensión de PHP para el servidor web Apache HTTP Server.

4.3. IDE

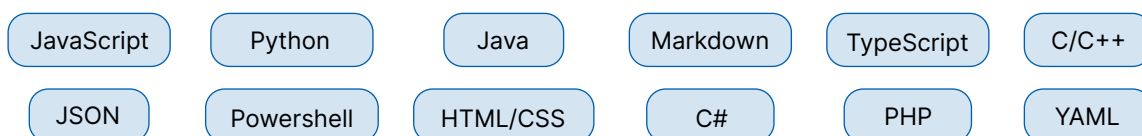
Utilizaremos la versión libre de Visual Studio Code (VSC) de Microsoft. VSCode puede ser instalado en Windows, Linux y MacOS.

IMPORTANTE

VSC no es el único IDE que existe. También podemos utilizar Eclipse o Netbeans para programar en Java. Dicho esto, para programar con PHP existen otras herramientas más completas, como phpStorm, de la empresa JetBrains, a la que se pueden solicitar licencias de uso educativo.

El IDE VSC, por defecto, admite lenguajes como HTML/CSS, Javascript y Typescript para el desarrollo web en entorno cliente, pero es extensible a otros lenguajes, como C++, C#, Python, Java y —evidentemente— PHP, para el desarrollo web en entorno servidor. Podremos crear guiones (*scripts*) con PowerShell o generar nuestra documentación o manuales con Markdown.

VS Code for



Diferentes lenguajes de programación admitidos por VSC.

Entre las principales características del editor, destacan su interfaz limpia, la paleta de órdenes —mediante la que podemos acceder a todas las opciones del editor—, la terminal integrada, el soporte de control de versiones, el uso de cursores múltiples, el resaltado de sintaxis y el autocompletado mediante IntelliSense. Además, es personalizable mediante un fichero en formato JSON, y permite ampliar la funcionalidad mediante extensiones. Aunque no se trate de un entorno integrado como tal, puede ofrecer prácticamente la misma funcionalidad que estos mediante estas extensiones.

4.4. xAMP

Disponemos de instaladores preparados para entornos de prueba como el XAMPP de Apache Friends (www.apachefriends.org), o los de la empresa Bitnami (bitnami.com/stacks), que cuenta con una versión diferente para cada tipo de sistema operativo:

- » LAMP, para sistemas operativo GNU/Linux.
- » MAMP, para macOS, de Apple.
- » WAMP, para Windows, de Microsoft.

Una forma más profesional de instalar LAMP en un servidor (con o sin entorno gráfico) es utilizar el metapaquete denominado **lamp-server^** de GNU/Linux. Es un *shell script* que contiene los comandos de instalación de las últimas versiones disponibles de estos paquetes:

- » Servidor Apache 2 (**paquete apache2**) de su fundación de software (apache.org).
 - » Sistema gestor de base de datos que queramos utilizar, tradicionalmente con MySQL (**paquetes mysql-server mysql-client libmysqlclient-dev**), o más recientemente con MariaDB (**paquetes mariadb-server mariadb-client**).
 - » Lenguaje de servidor PHP (**paquetes php libapache2-mod-php php-mysql php-common php-cli php-json php-opcache php-readline**) de la versión que esté disponible cuando se vaya a instalar.

A. Apache

Pasos que vamos a seguir para poder probar nuestras páginas dinámicas **.php** sobre el servidor web:

- » Debemos conocer dónde está ubicada la carpeta **htdocs**. En ella o en alguna de sus subcarpetas deberemos ir dejando nuestras páginas web con **.php** que vayamos desarrollando.
- » Utilizaremos el protocolo **HTTP (80)** o **HTTPS (443)** con sus puertos por defecto (los indicados entre paréntesis), en los que estará escuchando nuestro servidor web. Si no lo cambiamos durante la instalación, algún instalador suele utilizar el puerto 8080 o cualquier otro superior al 1024; siempre podemos personalizarlo.
- » Para probar si el código PHP que hemos escrito funciona correctamente deberemos acceder a la IP 127.0.0.1 o con el alias **localhost**, si estamos desarrollando en local. Podremos acceder remotamente, con la IP del equipo servidor o el nombre de dominio que le hayamos configurado.

Finalmente, el servidor realizará la transformación de la página web con el lenguaje de *scripting* PHP en otra página web nueva que solo contenga el lenguaje de marcas HTML, CSS o JavaScript.

Aunque durante la instalación podemos decidir cambiarlas, las rutas habituales de la carpeta **htdocs** son, por defecto, estas, junto a la versión (-8.1.6-0):

SISTEMA OPERATIVO	PLATAFORMA	RUTA DEL SISTEMA	RUTA DEL HOME
GNU/Linux	lamp-server^	/var/www/html	
GNU/Linux	Bitnami LAMP	/opt/lampstack/htdocs	~/lampstack/htdocs
macOS	Bitnami MAMP	/Applications/mampstack/apache2/htdocs	
Windows	Bitnami WAMP	C:\bitnami\wampstack\apache2\htdocs	
Windows	xAMPP	C:\xampp\htdocs	

PHP

Conocer la carpeta donde se encuentra el archivo **php.ini** es importante. A partir de este archivo, se configura el funcionamiento de PHP al arrancar. Además, este archivo php.ini contiene un gran número de directivas que se agrupan en secciones (que aparecerán entre corchetes: «[sección]»).

Cada sección, a su vez, tendrá un grupo de directivas asociadas. Estas directivas tienen esta sintaxis: **nombre_directiva = valor_directiva**, donde el valor de la directiva puede admitir valores de todo tipo.

Si no queremos que se aplique alguna de ellas, pero tampoco queremos eliminarla, podemos utilizar el símbolo punto y coma «;» al principio de la línea para comentarla.

Por último, es importante advertir que manipular el archivo php.ini sin conocimiento suficiente puede hacer inestable el intérprete de PHP y generar errores inesperados, difíciles de localizar y solventar.

IMPORTANTE: analiza previamente con detalle lo que necesitas cambiar.

La ruta de instalación del archivo **php.ini** suele estar en estas rutas:

SISTEMA OPERATIVO	PLATAFORMA	RUTA DEL SISTEMA	RUTA DEL HOME
GNU/Linux	lamp-server^	/etc	
GNU/Linux	Bitnami LAMP	/opt/lampstack/php/etc	~/lampstack/php/etc
macOS	Bitnami MAMP	/Applications/mampstack/php/etc	
Windows	Bitnami WAMP	C:\bitnami\lampstack\php	
Windows	xAMPP	C:\xampp\php	

Rutas o caminos donde encontrar el archivo php.ini.

Podemos ejecutar la función `phpinfo()` en un archivo `.php`. Solo ha de contener estos caracteres:

```
<?php phpinfo(); ?>
```

Al ejecutarlo, podemos ver la directiva **«Loaded Configuration File»**, que indica la ruta a `php.ini`.

Loaded Configuration File | **Applications/mampstack-8.1.6-0/php/etc/php.ini**



VÍDEOLAB

Para consultar el siguiente vídeo sobre instalación del servidor, escanea el código QR o [pulsa aquí](#).

UAX FP